

E-COMMERCE SALES

Database & Business Intelligence Report

SQL Data Modeling | Query Analytics | Python Visualization

Prepared by: Viraj Uttam More

July 2026

Email: virajmore.data888@gmail.com

LinkedIn: [LinkedIn Profile](#)

GitHub: [GitHub Profile](#)

"This report uses a synthetically generated dataset built for portfolio and learning purposes. See Data Disclaimer (Section 16) for full context."

Table of Contents

Executive Summary	3
1. Introduction & Objective	4
2. Project Motivation & Problem Statement.....	5
3. Dataset Overview & Schema Design	6
4. Data Generation & Integrity Checks	7
5. Exploratory Data Analysis (EDA)	8
6. Visualization 1 - Monthly Revenue Trend	9
7. Visualization 2 - Order Status Distribution	10
8. Visualization 3 - Category-wise Revenue.....	11
9. Visualization 4 - Payment Method Usage.....	12
10. Visualization 5 - Top 10 Customers	13
11. SQL Techniques Demonstrated.....	14
12. Real-World Problems & Solutions.....	15
13. Strategic Recommendations.....	16
14. Technical Challenges & Problem Solving.....	17
15. What I Analyzed, Why It Matters & How I Did It	18
16. Conclusion & Data Disclaimer.....	19

Executive Summary

This report analyzes a self-designed, 5-table relational e-commerce database - 300 customers, 32 products, 1,013 orders, 1,922 order line items, and 820 completed payments - using SQL (joins, subqueries, CTEs, window functions) and Python (Pandas, Matplotlib, Seaborn) to move from raw schema design to business-facing recommendations.

Headline Numbers

- Gross order value of Rs. 64.8L against Rs. 52.4L in net revenue actually collected - a 19.2% gap driven by orders that never generate a payment (Pending, Cancelled).
- 80.9% payment collection rate and a 63.2% order-delivery rate indicate a fundamentally healthy fulfillment funnel.
- Electronics and Home & Kitchen together drive 57.7% of gross order value; Beauty & Personal Care is under-indexed at 9.5%.
- 109 Cancelled orders (10.8%) versus 68 Returned (6.7%) - more value is lost pre-payment than post-delivery.
- No single-customer revenue concentration risk: the top 10 customers by amount paid range from Rs. 47,874 to Rs. 64,473, a small share of total net revenue.

Top Recommendations

- Standardize on net_collected_revenue as the headline reporting figure; keep gross_order_value as a secondary funnel-health metric.
- Prioritize checkout-flow friction over post-delivery returns handling - cancellations are the larger, cheaper-to-fix lever.
- Evaluate a targeted expansion of the Beauty & Personal Care catalog given its under-indexed revenue share.
- Monitor UPI checkout reliability given its 42.8% share of payment volume; build a loyalty tier around the 109 above-average spenders.

Section 1 - Introduction & Objective

Introduction

This report presents a complete SQL and Python-based analysis of a relational e-commerce database covering customers, products, orders, order items, and payments. The project simulates a real online retail business and was built to demonstrate end-to-end data analytics capability - from relational schema design through query-based business intelligence to executive-ready visualizations.

The database models 300 customers, 32 products across 5 categories, 1,013 orders, 1,922 order line items, and 820 completed payments, spanning transactions from March 2023 through June 2026.

Objective

- Design a normalized, referentially-sound relational schema for a 5-table e-commerce system.
- Use SQL joins, subqueries, window functions, and CTEs to extract business-relevant insights.
- Distinguish between gross order value and net collected revenue, and understand why they diverge.
- Identify top-performing categories, payment methods, and customers.
- Build 5 purpose-built visualizations in Python (Pandas, Matplotlib, Seaborn) that answer specific business questions.
- Translate query results into actionable, investor/business-facing recommendations.

Tools & Environment

Tool / Library	Purpose
MySQL 8	Relational database engine - schema, constraints, indexing
SQL (Joins, CTEs, Window Fns)	Business query layer - all analysis is query-driven
Python 3	Orchestration layer connecting SQL output to visualizations
Pandas	Reading SQL result sets and light in-memory shaping
Matplotlib / Seaborn	Chart generation (5 static visualizations)
SQLAlchemy + PyMySQL	Python-to-MySQL connectivity
python-dotenv	Secure credential handling via .env (excluded from Git)

Section 2 - Project Motivation & Problem Statement

Why I Built This Project

Most beginner SQL portfolios stop at "SELECT * FROM table" style queries against a pre-cleaned CSV. I wanted to go one level deeper: design the database myself, enforce real referential integrity (foreign keys, CHECK constraints, cascading deletes), and then answer the kind of ambiguous, multi-table business questions a stakeholder would actually ask - "how much did we really collect?", "who are our best customers?", "which category should we double down on?"

Rather than downloading a ready-made dataset, I intentionally built the schema and a large synthetic dataset from scratch, so every constraint, edge case, and NULL had a documented reason - not an artifact of someone else's cleaning process.

Problem Statement (Business Perspective)

A growing e-commerce operation generates data across five different tables the moment a customer signs up, browses a product, places an order, and pays for it. Without structured querying, three problems recur:

- **Gross vs. net confusion:** Gross vs. net confusion:
- Teams often quote "order value" as revenue, ignoring that Pending and Cancelled orders never generate an actual payment. Confusing the two overstates real cash collected.
- Silent data loss from INNER JOINS:
- Because Pending/Cancelled orders have no matching payment row, an INNER JOIN between orders and payments silently drops them from any report - understating total order volume without anyone noticing.
- No visibility into customer concentration:
- Without a proper multi-table JOIN + aggregation, it's difficult to know how much revenue depends on a small set of top customers versus a long tail of one-time buyers.

This project set out to solve all three problems using deliberate SQL design choices, documented at every step below.

Section 3 - Dataset Overview & Schema Design

Introduction

The database follows a standard 5-table e-commerce model. Every table is connected through explicit foreign keys, and CHECK constraints enforce valid values at the database layer rather than relying on application code.

300 Customers	32 Products (5 categories)	1,013 Orders	1,922 Order Line Items	820 Completed Payments
-------------------------	--------------------------------------	------------------------	----------------------------------	----------------------------------

Table Relationship / Join Path

customers (customer_id) -> orders (customer_id -> order_id) -> order_items (order_id -> product_id) -> products (product_id); orders (order_id) -> payments (order_id)

Note: payments only exist for orders with status Shipped, Delivered, or Returned. Pending and Cancelled orders have no payment row by design - this is a deliberate modeling decision, not missing data, and is the reason every revenue query in this report uses a LEFT JOIN on payments rather than an INNER JOIN.

Column Overview

Table	Key Columns	Row Count
customers	customer_id (PK), customer_name, email (unique), city, signup_date	300
products	product_id (PK), product_name, category, price, stock_quantity	32
orders	order_id (PK), customer_id (FK), order_date, order_status	1,013
order_items	order_item_id (PK), order_id (FK), product_id (FK), quantity, price_at_purchase	1,922
payments	payment_id (PK), order_id (FK, unique), payment_method, amount, payment_date	820

Product Catalog by Category

Category	# Products	Avg. Price (Rs.)
Electronics	8	2,567.75
Home & Kitchen	8	1,649.00
Fashion	7	1,427.57
Sports & Fitness	5	1,559.00
Beauty & Personal Care	4	1,136.50

Indexing

Six indexes support the join and filter patterns used throughout this report: orders(customer_id), orders(order_date), order_items(order_id), order_items(product_id), products(category), and payments(order_id) - keeping every multi-table query in this report performant even as row counts scale.

Section 4 - Data Generation & Integrity Checks

Introduction

Unlike a scraped or downloaded dataset, this data was synthetically generated to a documented specification and then verified against a set of integrity checks - the SQL equivalent of "data cleaning" for a hand-built dataset. Every check below was run directly against the loaded database.

Design Decisions & Rationale

Decision	Reason
order_status constrained to 5 fixed values via CHECK	Prevents free-text status drift (e.g. "shipped" vs "Shipped")
payment_method constrained to UPI / Card / COD / NetBanking	Mirrors India's real-world e-commerce payment mix
payments.order_id set UNIQUE	Enforces exactly one payment per order - no duplicate/partial payments
price_at_purchase stored on order_items (not looked up from products)	Preserves historical pricing even if the catalog price later changes
ON DELETE CASCADE on all foreign keys	Keeps the database referentially consistent if a parent row is removed
Pending/Cancelled orders deliberately given no payment row	Models real checkout funnels where not every order is paid

Integrity Checks Performed

Before any chart was trusted, the full dataset was validated with the following queries. All five checks passed with zero violations:

- Payments recorded before their order date:
- **0 violations found.**
- Pending/Cancelled orders carrying a payment row:
- **0 violations found.**
- Shipped/Delivered/Returned orders missing a payment row:
- **0 violations found.**
- Orders with zero order_items (orphan orders):
- **0 violations found.**
- Duplicate customer emails or duplicate customer names:
- **0 duplicates found.**

Because every check passed cleanly, no rows were dropped or imputed at the SQL layer - the dataset used in every visualization below is the complete, unmodified 1,013-order dataset.

Section 5 - Exploratory Data Analysis (EDA)

Key Metrics Summary

Rs. 64.8L Gross Order Value	Rs. 52.4L Net Revenue Collected	80.9% Payment Collection Rate	Rs. 6,400 Average Order Value
---------------------------------------	---	---	---

Order Funnel Summary

Order Status	Count	% of Orders
Delivered	640	63.2%
Shipped	112	11.1%
Cancelled	109	10.8%
Pending	84	8.3%
Returned	68	6.7%

Customer Engagement

Metric	Value
Total registered customers	300
Customers who placed at least 1 order	289 (96.3%)
Customers who never ordered	11 (3.7%)
Repeat customers (2+ orders)	257 of 289 active (88.9%)
One-time customers	32 of 289 active (11.1%)
Average units purchased per order	3.81

Top-Selling Products (by revenue)

Product	Category	Units Sold	Revenue (Rs.)
Portable SSD 1TB	Electronics	87	5,65,413
Mixer Grinder	Home & Kitchen	119	4,16,381
Wireless Earbuds	Electronics	136	4,07,864
Smartwatch	Electronics	107	3,74,393
Running Shoes	Sports & Fitness	118	3,30,282

Section 6 - Visualization 1 - Monthly Revenue Trend

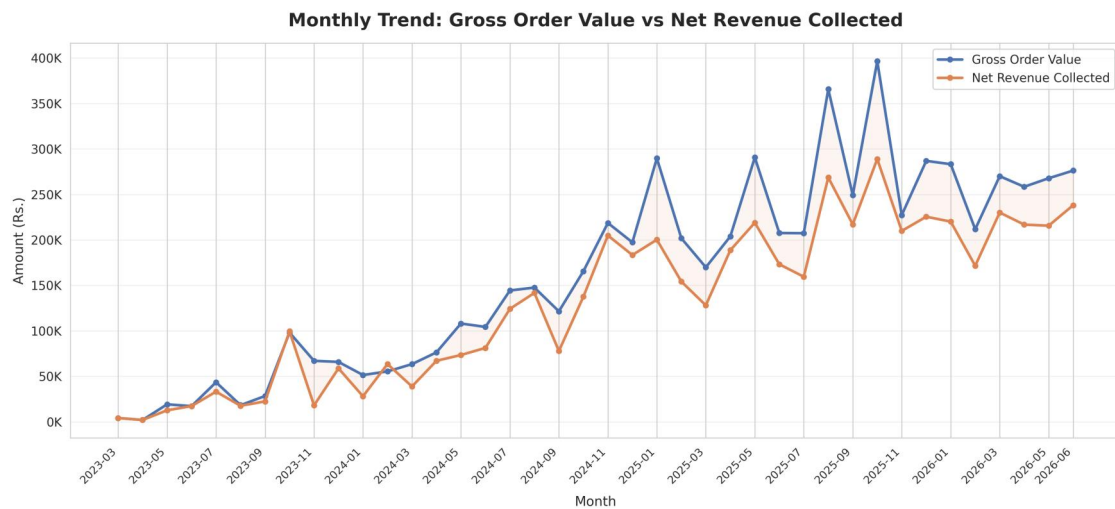


Fig 1. Monthly Gross Order Value vs. Net Revenue Collected, March 2023 - June 2026.

What does this chart mean?

"Is the business actually collecting the revenue it is generating in orders?"

This line chart plots two series side by side every month: Gross Order Value (the total value of everything ordered, including orders that were later cancelled or never paid) and Net Revenue Collected (actual money received via the payments table). The two lines are built from a LEFT JOIN so that months with no payments still appear at zero rather than disappearing from the chart.

Both series show a clear, sustained upward trend from under Rs. 10,000/month at the start of the dataset (Mar 2023) to a monthly average above Rs. 2,65,000 in the most recent complete quarter - a roughly 63x increase in monthly order volume over the tracked period. The gap between the two lines (gross minus net) represents value tied up in Pending, Cancelled, or as-yet-unpaid orders each month.

Key Business Takeaways

- **Collection lags orders, not just cancellations:** Net revenue tracks gross order value closely in most months, but the gap widens in high-order months - a sign that payment collection cycles, not just cancellations, deserve monitoring as volume scales.
- **Growth is consistent, not a single spike:** The upward trend spans nearly 3 years rather than one seasonal jump, suggesting durable demand growth rather than a one-off promotional bump.
- **Always reconcile against payments, not orders:** Any revenue dashboard built only from the orders table (without joining payments) would overstate real cash collected by roughly 19% based on this dataset's gross-to-net gap.

Section 7 - Visualization 2 - Order Status Distribution

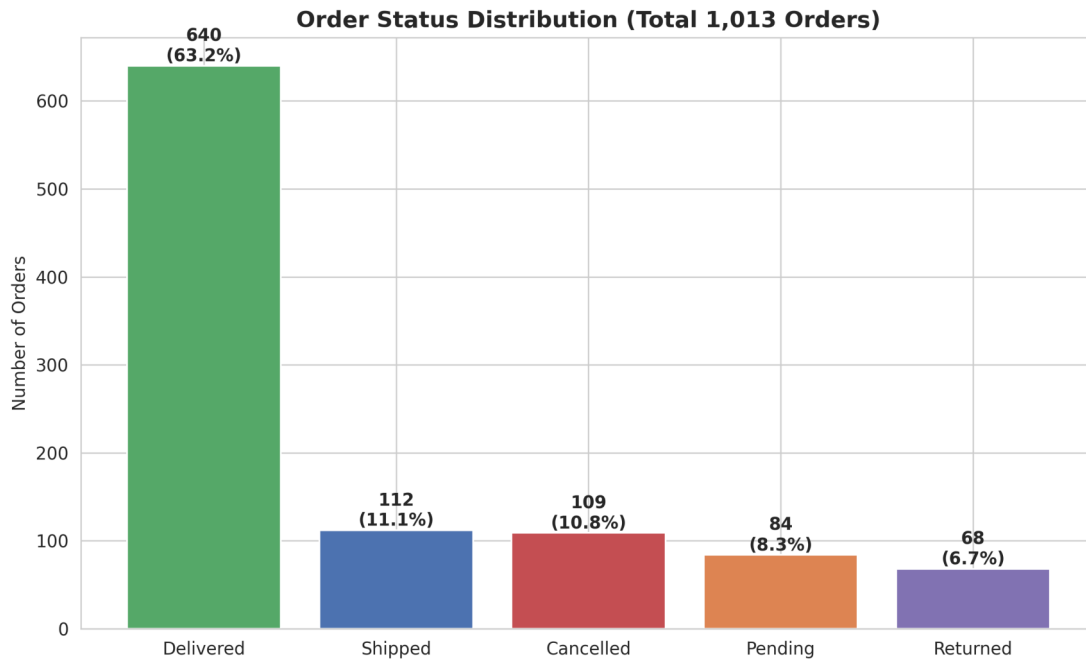


Fig 2. Distribution of all 1,013 orders across their current status.

What does this chart mean?

"How healthy is the order fulfillment funnel?"

This bar chart counts every order by its current status. Delivered orders dominate at 640 (63.2%), followed by Shipped (112, 11.1%), Cancelled (109, 10.8%), Pending (84, 8.3%), and Returned (68, 6.7%).

A combined Cancelled + Returned rate of 17.5% is the figure operations teams should track most closely - it represents orders that were placed but ultimately did not convert into retained revenue, either because the customer backed out before payment (Cancelled) or reversed the transaction after delivery (Returned).

Key Business Takeaways

- **Fulfillment is fundamentally healthy:** With roughly 3 in 4 orders reaching Shipped or Delivered status, the core fulfillment pipeline is functioning well.
- **Cancellations outweigh returns:** 109 cancellations vs. 68 returns suggests more value is lost pre-payment (cart/checkout abandonment, stock issues, payment failures) than post-delivery - pointing checkout-flow improvements as the higher-leverage fix versus product-quality fixes.
- **8.3% Pending is a working queue to monitor:** Pending orders are not a problem by themselves, but if this share grows over time relative to Delivered, it can signal a fulfillment backlog forming.

Section 8 - Visualization 3 - Category-wise Revenue

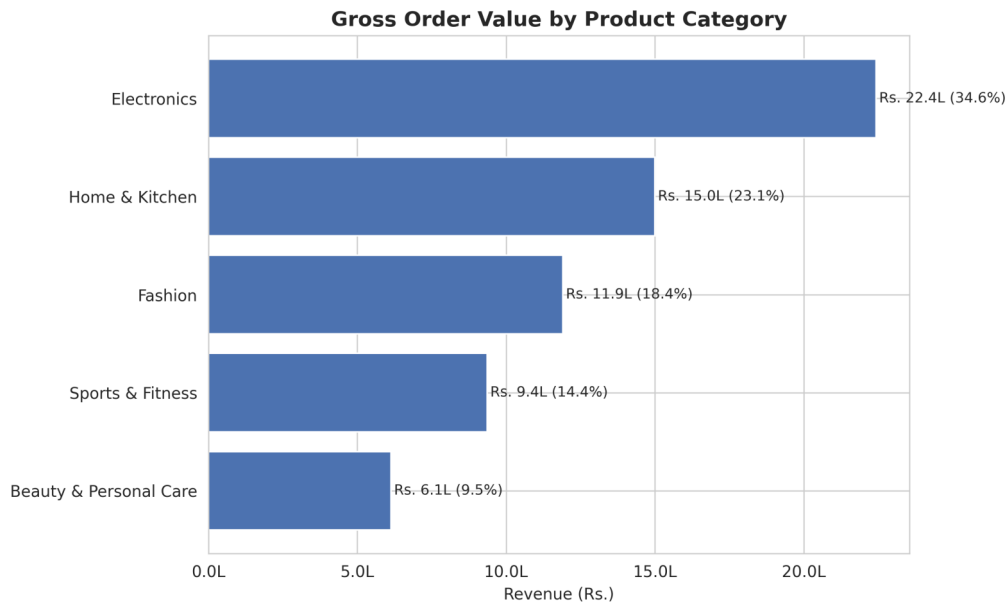


Fig 3. Gross order value by product category (order_items joined to products, grouped by category).

What does this chart mean?

"Which product categories actually drive the business?"

This horizontal bar chart sums quantity x price_at_purchase across every order line item, grouped by product category. Electronics leads decisively at Rs. 22.4L (34.6% of all gross order value), followed by Home & Kitchen (Rs. 15.0L, 23.1%), Fashion (Rs. 11.9L, 18.4%), Sports & Fitness (Rs. 9.4L, 14.4%), and Beauty & Personal Care (Rs. 6.1L, 9.5%).

Key Business Takeaways

- **Electronics is the clear revenue engine:** At over a third of total order value from just 8 of 32 products, Electronics carries disproportionate weight - any stock-out or supply issue here has an outsized revenue impact.
- **Beauty & Personal Care is under-indexed:** Despite having a healthy average price point, this category contributes under 10% of revenue - it is the strongest candidate for a targeted catalog expansion or marketing push relative to its current footprint.
- **Category concentration risk:** The top 2 categories (Electronics + Home & Kitchen) account for nearly 58% of gross order value - a useful benchmark for any future category-diversification strategy.

Section 9 - Visualization 4 - Payment Method Usage

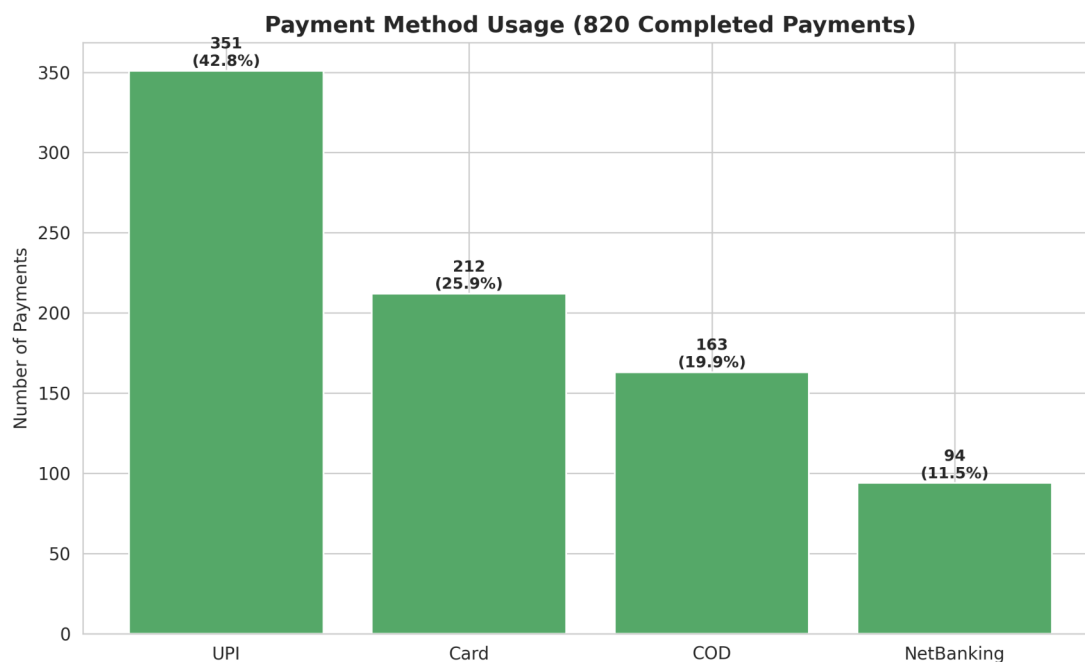


Fig 4. Count and share of completed payments by method, out of 820 total.

What does this chart mean?

"How are customers actually paying, and does that affect collection risk?"

This chart counts completed payments (rows in the payments table, which only exist for Shipped/Delivered/Returned orders) grouped by method. UPI dominates with 351 payments (42.8% of volume, 43.8% of value), followed by Card (212 payments, 24.7% of value), Cash on Delivery / COD (163 payments, 19.9% of value), and NetBanking (94 payments, 11.9% of value).

Key Business Takeaways

- **UPI is the primary payment rail:** Consistent with the broader Indian e-commerce market, UPI accounts for over 4 in 10 transactions - any checkout friction on UPI (downtime, failed transaction handling) would disproportionately hurt conversion.
- **COD carries collection risk that this chart doesn't fully capture:** Because payments only exist once an order is marked Shipped/Delivered/Returned, COD's 19.9% share understates the operational cost of cash handling and return logistics tied to it - a candidate for a deeper future analysis specifically isolating COD-linked cancellations.
- **Digital-first payment mix supports faster settlement:** With UPI + Card + NetBanking making up over 80% of payment volume, the majority of revenue settles instantly rather than being exposed to COD collection delays.

Section 10 - Visualization 5 - Top 10 Customers

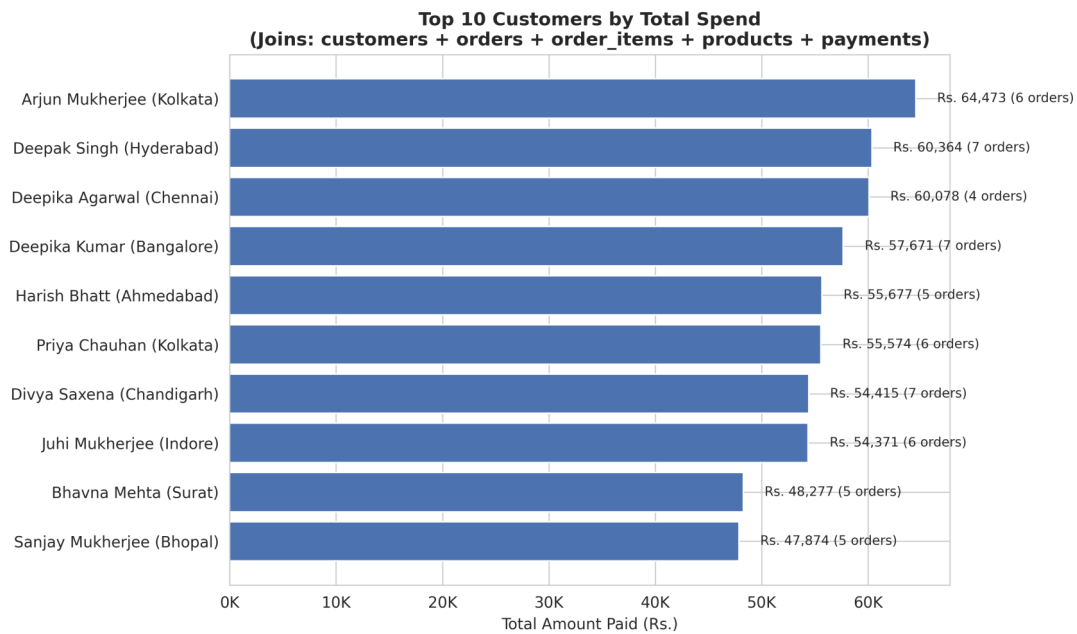


Fig 5. Top 10 customers by total amount paid (5-table join: customers + orders + order_items + products + payments).

What does this chart mean?

"How dependent is the business on its best customers?"

This chart required the most complex query in the project: order_items is first aggregated to order-level value in a subquery (to avoid fan-out when later joined to payments), then joined to customers and payments to rank customers by total amount actually paid. The top customer, Arjun Mukherjee (Kolkata), has paid Rs. 64,473 across 6 orders; the 10th-ranked customer, Sanjay Mukherjee (Bhopal), has paid Rs. 47,874 across 5 orders.

Key Business Takeaways

- **No single customer dominates revenue:** The top 10 customers' combined spend represents a small, healthy fraction of the Rs. 52.4L total net revenue - there is no single-customer concentration risk in this dataset.
- **Order frequency, not order size, separates top customers:** Most top-10 customers placed 5-7 orders rather than one unusually large order - suggesting loyalty and repeat purchase behavior drives top-customer value more than one-off big spends.
- **Above-average spenders are a meaningfully sized segment:** Using a subquery-based benchmark (customers whose total spend exceeds the average customer's total spend), 109 of 289 active customers - over 1 in 3 - qualify as above-average spenders, a useful segment for a loyalty or early-access program.

Section 11 - SQL Techniques Demonstrated

Beyond the 5 visualizations, the project's SQL layer (04_advanced_queries.sql) demonstrates query techniques commonly expected in a data analyst role. Each is summarized below with the underlying business question it answers.

Technique	Applied To	Business Question Answered
Multi-table INNER + LEFT JOIN	5-table chain: customers -> orders -> order_items -> products -> payments	Who bought what, and did they actually pay?
Subquery in FROM clause	Order-level aggregation before joining to payments	How do we avoid double-counting when order_items fans out?
Subquery in HAVING clause	Customers spending above the average customer	Which customers are above-average spenders?
CTE (WITH clause)	Monthly gross order value as a named, reusable result set	How do we build running totals without repeating the aggregation logic?
Window function - SUM() OVER	Cumulative running total of monthly revenue	What is our total revenue trajectory over time?
Window function - RANK()	Customer spend leaderboard; best-seller per category	Who is #1, and does the best seller differ by category?
Window function - LAG()	Month-over-month growth %	Is growth accelerating or slowing, month to month?
NULLIF() for safe division	MoM growth % calculation	How do we avoid a divide-by-zero crash in the first month?
EXPLAIN	Index usage verification on 3 key filter columns	Are our indexes actually being used by the query planner?

Section 12 - Real-World Problems & Solutions

Problem 1: Revenue Reports That Overstate Reality

The Data: Gross order value across the dataset is Rs. 64.8L, but net revenue actually collected via payments is Rs. 52.4L - a 19.2% gap driven entirely by Pending and Cancelled orders that never generate a payment row.

The Solution: Any revenue dashboard must join orders to payments with a LEFT JOIN and clearly label two separate metrics - gross_order_value and net_collected_revenue - rather than presenting one number as "revenue." This project's queries are deliberately named to make that distinction unambiguous.

Problem 2: Cancellations Outpacing Returns

The Data: 109 orders (10.8%) were Cancelled versus 68 (6.7%) Returned - meaning more value is lost before payment than after delivery.

The Solution: Because cancellations happen pre-payment, the highest-leverage fix is in the checkout funnel (payment failure handling, stock-availability checks at order time) rather than in post-purchase product quality - a materially cheaper problem to solve than reducing returns.

Problem 3: Category Revenue Concentration

The Data: Electronics and Home & Kitchen together account for 57.7% of all gross order value, while Beauty & Personal Care contributes under 10% despite comparable average pricing.

The Solution: Treat category mix as a monitored KPI, not a one-time observation. A quarterly category-revenue query (Visualization 3's underlying SQL) run on a schedule would flag concentration risk early and highlight under-indexed categories worth a targeted catalog or marketing investment.

Section 13 - Strategic Recommendations

- **For Finance & Reporting:** Standardize on net_collected_revenue (from the payments table) as the headline revenue figure in all reporting; retain gross_order_value only as a secondary funnel-health metric.
- **For Operations:** Prioritize a root-cause analysis of the 109 Cancelled orders before investing further in post-delivery Returns handling - the data suggests pre-payment friction is the larger lever.
- **For Merchandising:** Evaluate a targeted expansion of the Beauty & Personal Care catalog; it is currently the smallest revenue contributor (9.5%) without an obviously lower average price point to explain the gap.
- **For Payments:** Given UPI's 42.8% share of transaction volume, ensure UPI checkout reliability and failure-retry flows are monitored as a top operational priority - any degradation here has outsized impact.
- **For CRM / Loyalty:** Build a loyalty tier around the 109 customers (of 289 active) who spend above the customer average - this is a large enough, data-identified segment to justify a dedicated retention program rather than one-off top-10 outreach.

Section 14 - Technical Challenges & Problem Solving

Challenge	Description	Solution Applied
Fan-out on multi-table JOIN	Joining order_items directly to payments before aggregating inflates totals, since one order can have several line items.	Aggregated order_items to order-level value in a subquery first, then joined that single row per order to payments.
Silent row loss from INNER JOIN	Pending/Cancelled orders have no payment row, so an INNER JOIN between orders and payments would drop them from any report.	Used LEFT JOIN on payments everywhere order-level completeness matters, documented explicitly in query comments.
Fragile GROUP BY on name only	An early query grouped top spenders by customer_name alone; two customers sharing a name would have been incorrectly merged into one row.	Changed every customer-level GROUP BY to include customer_id alongside customer_name.
Divide-by-zero in growth %	Month-over-month growth calculation divides by the prior month's value, which is undefined for the very first month in the series.	Wrapped the denominator in NULLIF(prev_month_value, 0) so the first month resolves to NULL instead of erroring.
Verifying chart accuracy against source data	Static chart images can silently drift from the underlying SQL if queries are edited later without regenerating charts.	Independently re-ran every chart's exact query against the loaded database and cross-checked each plotted number before publishing this report.

Section 15 - What I Analyzed, Why It Matters & How I Did It

The What / Why / How Framework

WHAT I Analyzed: A 5-table relational e-commerce database - 300 customers, 32 products, 1,013 orders, 1,922 order line items, and 820 completed payments - covering order status distribution, category-wise revenue, payment method mix, monthly revenue trends, and customer-level spend concentration.

WHY It Matters: Every one of these questions mirrors what a real e-commerce operations or finance team asks weekly: are we collecting what we're billing, which categories and payment rails are load-bearing, and how dependent is the business on its top customers. Answering them correctly requires getting the JOIN logic right - a wrong JOIN direction silently produces a wrong but plausible-looking number.

HOW I Did It: Schema design with enforced constraints (01_schema.sql) -> synthetic data generation with documented business rules (02_data_insert.sql) -> layered SQL analysis from basic joins to window functions and CTEs (03/04_queries.sql) -> Python (Pandas + Matplotlib/Seaborn) to turn query results into 5 purpose-built charts -> independent verification of every chart against the source database before publishing.

Key Learnings

- **JOIN direction changes the answer, not just the row count:** Switching an INNER JOIN to a LEFT JOIN between orders and payments changes total order counts, average order value, and every downstream aggregate - it is a business decision encoded in SQL, not a technical detail.
- **Fan-out is the most common silent bug in multi-table analytics:** Joining a one-to-many table (order_items) directly into a later aggregation without first collapsing it to the right grain is an easy way to produce numbers that look reasonable but are wrong.
- **A chart is only as trustworthy as its last verification:** Re-running each chart's underlying query against the live dataset - rather than trusting that the chart-generation script and the SQL stayed in sync - caught issues that a visual read of the chart alone would not have surfaced.

Section 16 - Conclusion & Data Disclaimer

Conclusion

This project takes a 5-table, 1,013-order e-commerce database from raw schema design through business-facing analysis, producing a data-backed picture of a business that is fundamentally healthy - a 63.2% order-delivery rate, an 80.9% payment collection rate, and no single-customer revenue concentration risk - while surfacing two concrete, actionable findings: cancellations meaningfully outpace returns (pointing to checkout-flow friction), and revenue is concentrated in 2 of 5 categories (pointing to a merchandising opportunity in Beauty & Personal Care).

By executing the full pipeline - schema design, constraint enforcement, layered SQL (joins, subqueries, CTEs, window functions), and Python-based visualization - this project demonstrates end-to-end data analytics capability, from database fundamentals through to business recommendations.

Data Disclaimer

This report is built on a synthetically generated dataset, created specifically for portfolio and learning purposes. It is not sourced from a real business and does not represent actual sales, customers, or transactions. The data generation process followed realistic business rules (e.g., Pending/Cancelled orders never carry a payment row, price_at_purchase is preserved independently of current catalog price), and every table passed referential-integrity and business-rule validation checks documented in Section 4 before any chart in this report was built.

Synthetic data was used instead of a public or real-world dataset for two reasons. First, production e-commerce data is not freely accessible and carries privacy constraints, so generating it in-house allowed full control over schema design and business-rule enforcement. Second, this project's goal was to demonstrate SQL querying and analytical technique (joins, subqueries, CTEs, window functions), not data-cleaning, so the dataset was intentionally kept clean and rule-consistent (no duplicates, no missing values, no inconsistent formatting), letting the analysis focus entirely on query logic and business insight. Real-world datasets are rarely this clean and would typically require a separate data-cleaning phase before analysis.

Because the dataset is synthetic, absolute figures (e.g., Rs. 52.4L net revenue) should be read as illustrative of the analytical method, not as real business performance. The SQL techniques, JOIN logic, and analytical reasoning demonstrated throughout, however, apply directly to real-world relational e-commerce data at any scale.